A3-PS023-TC301

Minimal Two-Node Cloud Hypervisor VM Controller

Team Member 1

Prof. Dr. Baisakhi Das

baisakhi.das@iem.edu.in

Institute of Engineering and Management, Salt Lake, Kolkata

Team Member 2

Debarghaya Mitra

debarghaya@debarghaya.in

Institute of Engineering and Management, Salt Lake, Kolkata

Team Member 3

Swapnil Chakraborty

chakrabortyswapnil06@gmail.com

Institute of Engineering and Management, Salt Lake, Kolkata

AIORI Hackathon 3

October 14, 2026

Problem Description

A lightweight KVM cloud needs a thin controller, not a stack of heavy daemons.

1. Enterprise hypervisors use multi-daemon architectures whose overhead is too high for lightweight edge deployments.
2. Cloud Hypervisor boots one VM per process on Linux KVM with a minimal footprint, but has no notion of a cluster.
3. Challenge: a controller and two worker agents on two KVM hosts that create, start, stop and list VMs.
4. Placement must follow available host memory, and VM resource limits must be enforced.
5. API failures must be handled and VM processes cleaned up, with no orphaned VMs or leaks.

Background

What is needed to understand the problem and the proposed design.

1. KVM turns Linux into a hypervisor and is exposed to user space through /dev/kvm [6].
2. Cloud Hypervisor is a minimal VMM on KVM; it is controlled through a REST API served on a UNIX socket [1].
3. Each Cloud Hypervisor process runs exactly one VM, so “listing VMs” means asking every supervised process [1], [2].
4. Linux cgroup v2 can cap memory, CPU and process count, and kill every process of a VM at once [5].
5. Bearer tokens over TLS are a standard, light way to authenticate API callers [3], [4].

Requirements to Design

Each requirement in the statement and what it means for the design.

Requirement	Design implication
2 KVM hosts, preset networking	Verify /dev/kvm at startup; consume pre-made TAP or bridge
1 prepared VM image	Per-VM reflink copy of the base image
Authenticated controller to worker	Bearer token over TLS; mTLS as hardening
Memory-based placement	Workers report free RAM; controller reserves atomically
Enforced resource limits	Flavors, Cloud Hypervisor config and cgroup v2 caps
Clean failure handling	Supervision, rollback, reconciler, cgroup kill

Where the Difficulty Hides

The statement looks simple; these are the parts that decide the result.

1 **Orphaned VMs**

A worker crash can leave cloud-hypervisor processes eating RAM.

2 **Partial failures**

vm.create succeeds, vm.boot fails: leaked disk and process.

3 **Placement races**

Two requests see the same free memory and overcommit a host.

4 **State divergence**

Controller inventory and worker reality drift after restarts.

Assumptions and Non-goals

What we take as given, and what we deliberately leave out.

Assumptions

- Linux with KVM, cgroup v2 and a cloud-hypervisor binary
- One prepared image at the same path on both hosts
- Preconfigured bridge or TAP; controller reaches both workers
- Workers run as root or with CAP_NET_ADMIN

Non-goals

- Live migration and distributed storage
- Browser VNC and an image marketplace
- A new hypervisor
- Also ours: HA controller, RBAC, CPU overcommit

Literature & RFC Review

Only the most relevant standards and studies, and how each is used.

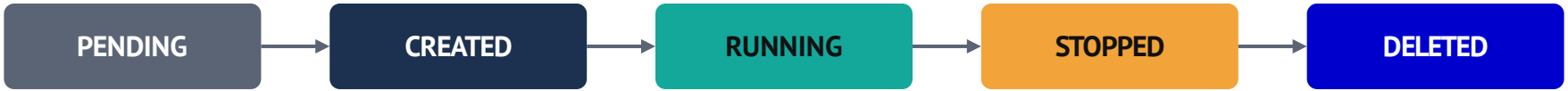
Source	Key idea	Used in our design
[1] Cloud Hypervisor API	REST over a UNIX socket: vm.create, vm.boot, vm.shutdown, vm.delete	Worker lifecycle integration
[2] Firecracker (NSDI 2020)	Minimal VMM; one process and one API socket per micro-VM	Process-per-VM supervision
[3] RFC 6750	Bearer tokens in the Authorization header	Controller and worker API auth
[4] RFC 8446	TLS 1.3 protects credentials in transit	Encrypted controller-worker link
[5] Linux cgroup v2	memory.max, cpu.max, pids.max, cgroup.kill	Limits and leak-free teardown
[6] KVM	In-kernel hypervisor exposed as /dev/kvm	Hardware virtualisation

Proposed Solution

One controller, two worker agents, one Cloud Hypervisor process per VM.

1. Controller: authenticated REST API, SQLite inventory, memory-based placement and a periodic reconciler.
2. Worker agent (per host): spawns cloud-hypervisor in its own cgroup, drives vm.create, vm.boot, vm.shutdown and vm.delete, reports free memory.
3. Security: bearer token over TLS on both API tiers; VM sockets are owned by the agent only.
4. Cleanup: cgroup kill, parent-death signal, write-ahead metadata and undo lists leave no orphans.

VM lifecycle



Cloud Hypervisor Integration

The management API calls each worker agent relies on.

1 : 1

One cloud-hypervisor process per VM, driven over its own UNIX socket. “List VMs” really means “ask each supervised process”.

Endpoint (/api/v1/)	Purpose
vmm.ping GET	Health and version
vm.create PUT	Define VM, do not boot
vm.boot PUT	Boot a created VM
vm.info GET	Config and state
vm.shutdown PUT	Stop the guest
vm.delete PUT	Remove definition

Two Small APIs

One operator-facing API on the controller, one internal API on each worker.

Controller API (operators)

POST /v1/vms	create
GET /v1/vms	list
POST /v1/vms/{id}/start	start
POST /v1/vms/{id}/stop	stop
DELETE /v1/vms/{id}	delete
GET /v1/workers	capacity

Worker API (controller only)

POST /v1/vms	spawn + define
GET /v1/vms	live state
POST /v1/vms/{id}/start	boot
POST /v1/vms/{id}/stop	stop + reap
DELETE /v1/vms/{id}	clean up
GET /v1/capacity	free memory

Auth: Authorization: Bearer token, constant-time compare, TLS, separate credentials per tier.

Memory-based Placement

Pick a host with enough free RAM, and never overbook.

1. Ask each healthy worker for capacity
2. Drop workers without enough free RAM
3. Pick the host with the most free memory
4. Reserve it in SQLite under a lock
5. Call the worker; roll back on failure

Effective free memory

min(MemAvailable − reserve,
total − reserve − Σ guest RAM)

Taking the minimum of the real and committed views guards against guest pages not yet faulted in.

Three Layers of Limits

A misbehaving request or guest cannot harm the host.

1

Admission control

Only allowed vCPU and RAM sizes pass; the worker re-validates.

2

Hypervisor config

boot_vcpus equals max_vcpus; memory.size exactly as granted.

3

cgroup v2 caps

memory.max = RAM + 128 MiB, cpu.max = vCPUs × 100%, pids.max.

small 1 vCPU · 512 MiB

medium 2 vCPU · 1 GiB

large 4 vCPU · 2 GiB

Failure Handling

Every failure ends in a clean, known state.

Failure	Response
CH fails to start	Kill cgroup, delete disk copy, mark FAILED
vm.create or vm.boot error	Roll back through the undo list
Graceful shutdown hangs	vmm.shutdown, then SIGTERM, then cgroup.kill
Worker unreachable	Mark unhealthy, stop placing, VM becomes UNKNOWN
Agent or CH crash	Startup reconcile: adopt or kill; mark FAILED
Controller dies mid-create	Reconciler resolves leftover PENDING rows

No Orphans, by Construction

Five mechanisms that make leaks practically impossible.

1	**cgroup per VM**	cgroup.kill ends every descendant at once
2	**Parent-death signal**	PR_SET_PDEATHSIG: VMs die with the agent
3	**Write-ahead metadata**	meta.json written before the spawn
4	**Reconciler loop**	Compares inventory, metadata, live processes
5	**Timeouts + undo lists**	Every call bounded; every step reversible

Optimization Proposed

Before (naive design) versus after (proposed design).

Aspect	Before: naive approach	After: proposed optimization
Placement	Read MemAvailable only; races overcommit	min(real, committed) plus a reservation lock
Teardown	Kill stored PID; children can leak	cgroup.kill, death signal and reconciler
Limits	Recorded only, not enforced	Flavors, CH config and cgroup v2 caps
Disk	Full image copy per VM	Reflink copy (cp --reflink=auto)
Failures	Ad-hoc error handling	Undo list and rollback per operation

Solution Architecture & Design

Visual: controller, two workers and their VM processes.

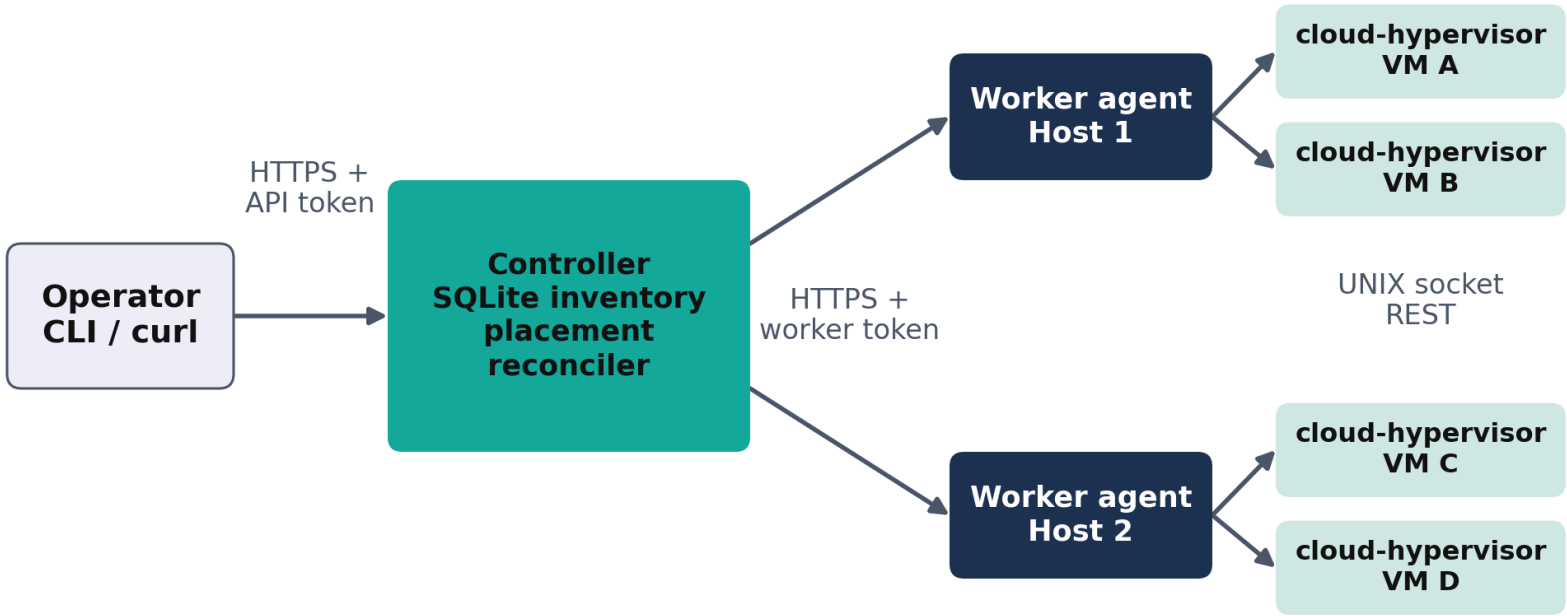


Figure: System architecture (deployment) diagram

Data Flow Diagram

Visual: how a request flows through the system.

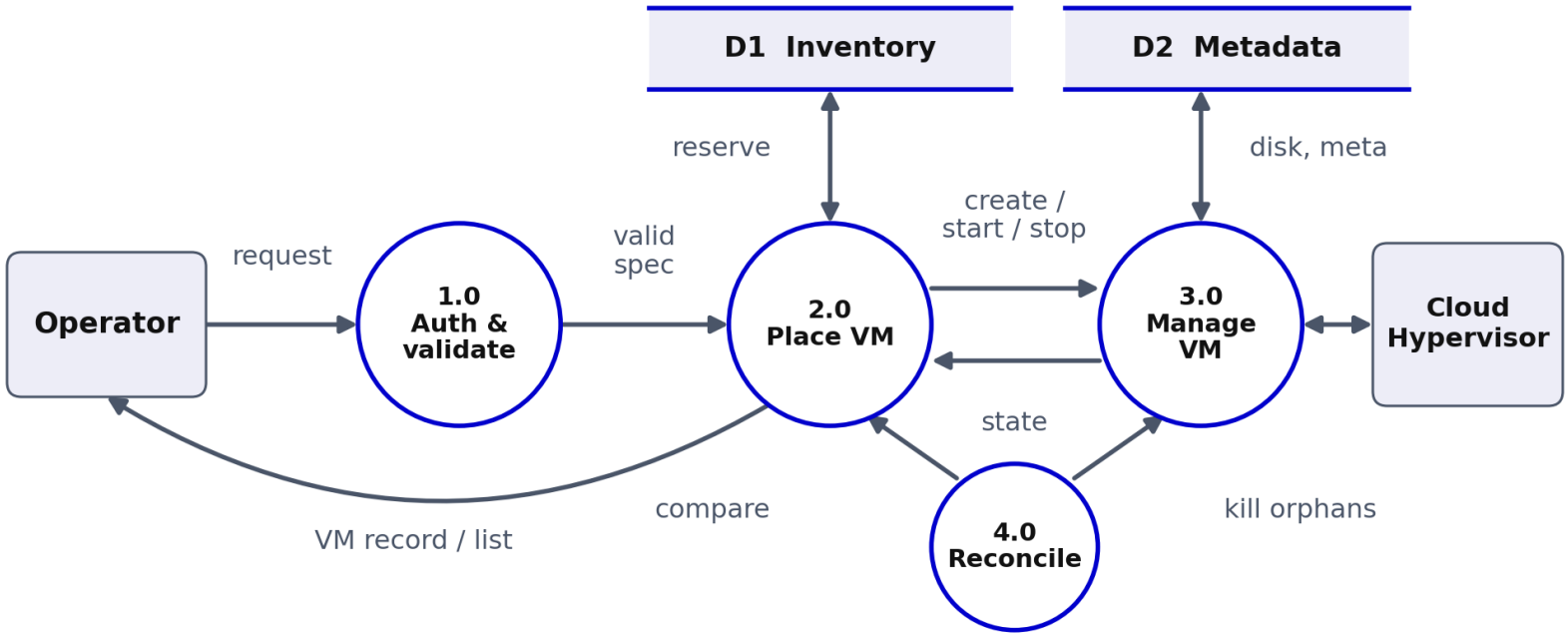


Figure: Data Flow Diagram (DFD)

Pseudocode: Create Path

Controller placement and worker spawn, with rollback on any error.

Controller

```
create_vm(req):
  authenticate(req)
  spec = validate(req)
  with placement_lock:
    w = best_fit(workers, spec.mem)
    if not w: raise NoCapacity
    vm = db.insert(PENDING, w)
  try:
    w.post("/v1/vms", vm.id, spec)
    db.set(vm, CREATED)
  except:
    w.try_delete(vm.id)
    db.set(vm, FAILED); raise
```

Worker agent

```
agent_create(id, spec):
  check_limits(spec)
  write_meta(id, "creating")
  try:
    disk = reflink(BASE_IMG)
    cg = cgroup(mem_max, cpu_max)
    proc = spawn_ch(sock, in=cg)
    wait_ping(sock, 5s)
    ch.put("vm.create", cfg(spec))
    write_meta(id, "created")
  except:
    run_undo(); remove_meta(id)
    raise
```

Test Strategy

Unit, lifecycle, two-host placement and failure-injection tests.

Unit (no KVM)

Placement, validation, auth, state machine, fake CH socket.

Lifecycle

Create, start, stop, delete; nothing left behind.

Two-host placement

Fill Host 1, spill to Host 2, reject oversize.

Failure injection

kill -9 CH or agent, block worker port, kill controller.

Leak check after every test: processes, cgroups, sockets and disks match inventory

Demonstration and Deliverables

What the mentor sees, and what we hand over.

Live demo (5–7 min)

1. Workers healthy; place two VMs by memory
2. Start both; show serial output or ping
3. Reject a VM too large for either host
4. Kill a CH process; show FAILED, memory freed
5. Stop, delete, then run the leak check

Deliverables

- ✓ Private repo A3-PS023-TC301
- ✓ Add aiori-hackathon; transfer ownership
- ✓ PDF report in the repo, given structure
- ✓ Demo with pseudocode for the mentor

Deployment Timeline

Official hackathon milestones, and our build plan inside them.



14 Oct	16 Oct	16–31 Oct	31 Oct–4 Nov	5 Nov	24–30 Nov	17 Dec
Level 1 steps 1–7 complete	Level 2 teams announced	Mentoring sessions and steps 8–10	Evaluators connect with teams	Regional teams announced	Regional hackathon: Bengaluru, Pune, Kolkata, Noida	Finals: 8 teams at IIT Jammu

Our build plan (inside the 16–31 Oct mentoring window)

- Incremental delivery: single-host agent, then controller, then reconciler, then tests, report and demo.
- Mentoring sessions review each milestone; each one ends with a leak check on both hosts.
- Cloud Hypervisor version pinned so the API schema stays stable.

Bibliography I

[1] Cloud Hypervisor Project. “Cloud Hypervisor REST API documentation.” GitHub, docs/api.md. github.com/cloud-hypervisor/cloud-hypervisor

[2] Alexandru Agache et al. “Firecracker: Lightweight Virtualization for Serverless Applications.” In: Proc. USENIX NSDI (2020).

[3] Michael B. Jones and Dick Hardt. “The OAuth 2.0 Authorization Framework: Bearer Token Usage.” RFC 6750, IETF (2012).

[4] Eric Rescorla. “The Transport Layer Security (TLS) Protocol Version 1.3.” RFC 8446, IETF (2018).

[5] The Linux Kernel Documentation. “Control Group v2.” docs.kernel.org/admin-guide/cgroup-v2.html.

[6] Avi Kivity et al. “kvm: the Linux Virtual Machine Monitor.” In: Proceedings of the Linux Symposium (2007).

Thank You for Your Attention!